

MIPS

* 32 bit registers

Instruction + dest + source1 + source2

- save registers (\$30 → \$31)
- temporary registers (\$t0 - \$t9)
- zero registers (\$zero)

$a = b + c \rightarrow \$s2$
 $\downarrow \quad \downarrow$
 $\$s0 \quad \$s1$

add \$s0, \$s1, \$s2

$p = (a+b) - (c+d)$

add \$t0, \$s1, \$s2

add \$t1, \$s3, \$s4

sub \$s0, \$t0, \$t1

$a = b + c + d \rightarrow \$s3$
 $\downarrow \quad \downarrow \quad \downarrow$
 $\$s0 \quad \$s1 \quad \$s2$

add \$t0, \$s2, \$s3

add \$s0, \$s1, \$t0

* How to do MOD

$A = b \times 8 \rightarrow$ will do andi with number -1

andi \$s0, \$s1, 7

— x —

Spilling Registers

(lw) register, offset (base address)
 ↳ also sw

Ex:

```

int arr_sum(int arr[], int n) {
    int sum = 0, a = 10, b = -7, c = 12;
    for (int i = 0; i < n; i++) {
        if (arr[i] <= 0) {
            b = (a < 15) && (b != 0);
        }
        else {
            b = ((~15 + c) & 1);
        }
        sum += arr[i + b];
    }
    return sum;
}

```

arr_sum:

add \$v0, \$zero, \$zero sum = 0

addi \$t0, \$zero, 10 a = 10

addi \$t1, \$zero, -7 b = -7

addi \$t2, \$zero, 12 c = 12

addi \$t3, \$zero, 0 i = 0

loop_start:

slt \$t4, \$t3, \$a1 if (i < n)

beq \$t4, \$zero, loop_end i > n ⇒ loop_end

sll \$t3, \$t3, 2 i = i * 4

add \$t5, \$t3, \$a0 i + base of arr

lw \$t6, 0(\$t5) read arr[i]

slt \$t7, \$zero, \$t6 if (0 < arr[i])

bneq \$t7, \$zero, else 0 < arr[i] ⇒ else

addi \$t8, \$zero, 15 save 15
 slt \$t7, \$t0, \$t8 check $a < 15$
 sub \$t9, \$t1, \$t2 $b - c$
 sltu \$t9, \$t9, \$zero check $b - c = 0$
 and \$t1, \$t7, \$t9 $b = (a < 15) \&\& (b \neq c)$
 j end-if

else:

addi \$t8, \$zero, 15 saves 15
 nor \$t8, \$t8, \$zero ~ 15
 add \$t8, \$t8, \$t2 $\sim 15 + c$
 andi \$t8, \$t8, 1 $(\sim 15 + c) \& 1$

end-if:

add \$t8, \$t3, \$t1 $i = i + b$
 sll \$t8, \$t8, 2 $i = i * 4$
 add \$t8, \$a0, \$t8 $arr[i]$
 add \$v0, \$v0, \$t8 $sum = sum + arr[i]$
 addi \$t3, \$t3, 1 $i++$
 j loop-start

loop-end:

jra \$ra

int superFunc (int ~~array~~ ^{\$a0}[], int size, int k) {
 int x = k, sum = 0, mul = 1;
 while (x < size - 1) {
 int j = array[x + 1];
 while (j != array[x]) {
 mul *= 5;
 j--;
 sum += (mul + j);
 array[x - 1] = sum;
 x += 2;
 }
 if (sum < mul)
 return sum;
 else
 return sum + mul;
 }
 }

superFunc:

addi \$s0, \$a2, 0 $x = k$

addi \$s1, \$zero, 0 $sum = 0$

addi \$s2, \$zero, 1 $mul = 1$

outer_loop:

addi \$t0, \$a1, -1 $size - 1$

slt \$t1, \$s0, \$t0 $x < size - 1$

beq \$t1, \$zero, final_work outer loop ends

addi \$t0, \$s0, 1 $x + 1$

sll \$t0, \$t0, 2 $x + 1 = (x + 1) * 4$

addi \$t0, \$t0, \$a0 $arr[x + 1]$

lw \$t1, 0(\$t0) load $arr[x + 1]$

add \$t2, \$t1, \$zero $j = arr[x + 1]$

inner_loop:

addi \$t0, \$s0, 0 x

sll \$t0, \$t0, 2 $x = x * 4$

addi \$t0, \$t0, \$a0 $arr[x]$

lw \$t1, 0(\$t0) load $arr[x]$

beq \$t2, \$t1, end_inner $j == arr[x]$ check

addi \$t3, \$s2, 0

sll \$s2, \$s2, 2 $mul = mul * 4$

add \$s2, \$s2, \$t3 $mul *= 5$

addi \$t2, \$t2, -1 $j--$

j inner_loop

end_inner_loop:

add \$t3, \$s2, \$t2 $mul + j$

add \$s1, \$s1, \$t3 $sum += (mul + j)$

addi \$t0, \$s0, -1 $x - 1$

sll \$t0, \$t0, 2 $x - 1 = (x - 1) * 4$

addi \$t0, \$t0, \$a0 $arr[x - 1]$

lw \$t1, 0(\$t0) load $arr[x - 1]$

```

add $t1, $s1, $zero
sw $t1, 0($t0)     $arr[x-1] = sum$ 
addi $s0, $s0, 2     $x += 2$ 
j outer-loop

```

final-work:

```

slt $t4, $s1, $s2
beq $t4, $zero, ELSE
addi $v0, $s1, $zero
jra $ra

```

ELSE:

```

add $t4, $s1, $s2
addi $v0, $t4, $zero
jra $ra

```

—— x ——

main:

```

addi $sp, $sp, -80
add $s0, $zero, $sp     $\text{base of } a[]$ 
addi $s1, $sp, 40     $\text{base of } b[]$ 
addi $s2, $zero, 1     $x = 1$ 
addi $s3, $zero, 32     $i = 32$ 

```

do-while:

```

sll $t0, $s3, 1     $2i$ 
sll $t0, $t0, 2     $2i * 4$  byte offset
add $t1, $t0, $s0     $a[2i]$ 
lw $t2, 0($t1)    load  $a[2i]$ 
addi $t0, $s3, 1     $i++$ 
sll $t0, $t0, 1     $2(i+1)$ 
sll $t0, $t0, 2     $2(i+1) * 4$  byte
add $t1, $t0, $s1     $b[2(i+1)]$  offset
lw $t3, 0($t1)    load  $b[2(i+1)]$ 
bne $t3, $t2, else-if    go to else if

```

```

int main() {
    int x=1, i=32, a[10], b[10];
    do {
        if (a[2i] == b[2(i+1)]) {
            x += a[2i] - b[2(i+1)];
            break;
        }
        else if (a[2i] & b[2(i+1)]) {
            i += x;
            continue;
        }
        else {
            i = i - 3;
        }
        i = i + 1;
    } while (i < 4);
    return 0;
}

```

sub \$t3, \$t2, \$t3 $a[2i] - b[2(i+1)]$
add \$s2, \$s2, \$t3 $x += a[2i] - b[2(i+1)]$
j end-loop go to end

else-if:

and \$t4, \$t2, \$t3 $a[2i] \& b[2(i+1)]$
beq \$t4, \$zero, ELSE go to else
add \$s3, \$s3, \$s2 $i += x$
j loop-condition go to check while

ELSE:

addi \$s3, \$s3, -3 $i = i - 3$
addi \$s3, \$s3, 1 $i = i + 1$

loop-condition:

andi \$t5, \$s3, 3 $i \% 4$ check
bne \$t5, \$zero, do-while

end-loop:

add \$v0, \$zero, \$zero
addi \$sp, \$sp, 80
jra \$ra

MIPS Assembly Translation Exam

Computer Organization & Architecture

Time: 3 hours Total: 19 Questions Full Marks: 190 Difficulty: Medium–Hard

Instructions:

- Write clear, correctly-labeled MIPS assembly for each question.
- Assume standard MIPS calling conventions unless a register mapping is given in comments.
- Where register assignments are provided in comments, you **must** use them.
- Show all labels, branches, and stack management explicitly for recursive/nested-call functions.
- Partial credit is given for correct logic even with minor syntax errors.

Section A Nested Control Flow & Arithmetic

Question 1 (10 marks)

Translate the following C code into MIPS assembly.

```
int collatzSteps(int n) {           // n = $a0
    int steps = 0;                  // steps = $t0
    while (n != 1) {
        if ((n & 1) == 0) {
            n = n / 2;
        } else {
            n = 3 * n + 1;
        }
        steps++;
        if (steps > 1000) {
            return -1;
        }
    }
    return steps;
}
```

Answer:



Question 2 (10 marks)

Translate the following C code into MIPS assembly.

```

int nestedLoopSum(int n, int m) {           // n = $a0, m = $a1
    int total = 0;                          // total = $s0
    int i, j;                              // i = $s1, j = $s2
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if ((i + j) % 2 == 0) {
                total += i * j;
            } else {
                total -= (i + j);
            }
        }
    }
    return total;
}

```

Answer:

nestedLoopSum:

```

    addi $sp, $sp, -20
    sw $a0, 16($sp)
    sw $a1, 12($sp)
    sw $s0, 8($sp)
    sw $s1, 4($sp)
    sw $s2, 0($sp)

```

outerLoop:

```

    addi $t0, $s1, 0    i=0
    slt $t1, $t0, $a0   i<n
    beq $t1, $zero, end-loop
    add $s1, $s1, 1     i++

```

innerLoop:

```

    addi $t2, $s2, 0    j=0
    slt $t3, $t2, $a1   j<m
    beq $t3, $zero, outerLoop
    add $t4, $s1, $s2    i+j
    andi $t4, $t4, 1     (i+j)%2

```

bne \$t4, \$zero, else

mul \$t4, \$s1, \$s2 i*j

add \$s0, \$t4, \$s0 total += i*j

add \$s2, \$s2, 1 j++

j innerLoop

else:

add \$t4, \$s2, \$s1 i+j

add \$s0, \$t4, \$s0 total += (i+j)

add \$s2, \$s2, 1 j++

j innerLoop

end-loop:

add \$v0, \$s0, \$zero

lw \$a0, 16(\$sp)

lw \$a1, 12(\$sp)

lw \$s0, 8(\$sp)

lw \$s1, 4(\$sp)

lw \$s2, 0(\$sp)

addi \$sp, \$sp, 20

jr \$ra



Question 3 (10 marks)

Translate the following C code into MIPS assembly.

```
int main() {
    int x = 3, y = 2;          // x = $s0, y = $s1
    int limit = 50;            // limit = $s2
    int flips = 0;             // flips = $s3

    do {
        if ((x & 1) == 0) {
            x += y;
            y += x;
        } else if ((y | 1) > y) {
            y += x;
            x += y;
            flips++;
        } else {
            x += 1;
            y += 1;
        }
    } while (x + y < limit && flips < 10);

    return flips;
}
```

Answer:



Question 4 (10 marks)

Translate the following C code into MIPS assembly.

```
int classify(int n, int mode) {      // n = $a0, mode = $a1
    int result;                     // result = $t0
    switch (mode) {
        case 0:
            result = (n % 3 == 0) ? 100 : 200;
            break;
        case 1:
            switch (n % 4) {
                case 0: result = 10; break;
                case 1: result = 20; break;
                default: result = 30; break;
            }
            break;
        default:
            result = -1;
    }
    return result;
}
```

Answer:



Section B Arrays, Pointers & Pointer Arithmetic

Question 5 (10 marks)

Translate the following C code into MIPS assembly.

```
int weightedSum(int arr[], int n) {           // arr = $a0, n = $a1
    int total = 0;                           // total = $t0
    int i;                                   // i = $t1
    for (i = 0; i < n; i++) {
        if (arr[i] < 0) {
            total -= arr[i] * 2;
        } else if (arr[i] > arr[0]) {
            total += arr[i] * 3;
        } else {
            total += arr[i];
        }
    }
    return total;
}
```

Answer:



Question 6 (10 marks)

Translate the following C code into MIPS assembly.

```
void insertSorted(int arr[], int n, int key) {  
    // arr = $a0, n = $a1, key = $a2  
    int i = n - 1;      // i = $t0  
    while (i >= 0 && arr[i] > key) {  
        arr[i + 1] = arr[i];  
        i--;  
    }  
    arr[i + 1] = key;  
}
```

Answer:



Question 7 (10 marks)

Translate the following C code into MIPS assembly.

```
int* findFirstGreater(int *start, int *end, int target) {  
    // start = $a0, end = $a1, target = $a2  
    int *p = start;      // p = $t0  
    while (p != end) {  
        if (*p > target) {  
            return p;  
        }  
        p = p + 1;  
    }  
    return end;  
}
```

Answer:



Question 8 (10 marks)

Translate the following C code into MIPS assembly.

```
void compressString(char *s) {           // s = $a0
    char *read = s;                      // read = $t0
    char *write = s;                     // write = $t1
    while (*read != '\0') {
        if (*read != ' ') {
            *write = *read;
            write++;
        }
        read++;
    }
    *write = '\0';
}
```

Answer:



Section C Recursion, Mutual Recursion & Function Calls

Question 9 (10 marks)

Translate the following C code into MIPS assembly.

```
void recursiveCopy(char dest[], char src[], int i, int j) {  
    if (src[i] == '\0') {  
        dest[j] = '\0';  
        return;  
    }  
    recursiveCopy(dest, src, i + 1, j - 1);  
    dest[j] = src[i];  
}
```

Answer:

```

recursiveCopy;
    addi $sp, $sp, -20
    sw $ra, 16($sp)
    sw $a0, 12($sp)
    sw $a1, 8($sp)
    sw $a2, 4($sp)
    sw $a3, 0($sp)

    add $t0, $a1, $a2    src = i
    lb $t1, 0($t0)
    bne $t1, $zero, recursive
    add $t1, $a0, $a3    dest = j
    sb $zero, 0($t1)    dest[j] = '\0'
    jra $ra

```

```

recursive8
    addi $a2, $a2, 1    i++
    addi $a3, $a3, -1    j--
    jal recursiveCopy
    lw $a0, 12($sp)
    lw $a1, 8($sp)
    lw $a2, 4($sp)
    lw $a3, 0($sp)

    lb $t3, 0($t0)    src[i]
    sb $t3, 0($t1)    dest[j] = src[i]

```

Question 10 (10 marks)

Translate the following C code into MIPS assembly.

```
int power(int base, int exp) {      // base = $a0, exp = $a1
    if (exp == 0) {
        return 1;
    }
    if (exp % 2 == 0) {
        int half = power(base, exp / 2);
        return half * half;
    }
    return base * power(base, exp - 1);
}
```

Answer:

```

power:
    bne $a1, $zero, if
    addi $v0, $zero, 1
    jra $ra

if:
    andi $t0, $a1, 1    exp %2
    bne $t0, $zero, end
    addi $sp, $sp, -12
    sw $ra, 8($sp)
    sw $a0, 4($sp)
    sw $a1, 0($sp)
    sra $a1, $a1, 1    exp /2
    jal power
    add $t1, $v0, $zero    half = power (base, exp/2)
    mul $v0, $t1, $t1
    lw $a0, 4($sp)
    lw $a1, 0($sp)
    jra $ra

end:
    addi $sp, $sp, -16
    sw $ra, 12($sp)
    sw $a0, 8($sp)
    sw $a1, 4($sp)
    sw $v0, 0($sp)
    addi $a1, $a1, -1    exp -1
    jal power
    lw $t2, 0($v0)
    mul $v0, $t2, $a0
    jra $ra

```

Question 11 (10 marks)

Translate the following C code into MIPS assembly.

```
int isOdd(int n);      // forward declaration

int isEven(int n) {    // n = $a0
    if (n == 0) return 1;
    return isOdd(n - 1);
}

int isOdd(int n) {     // n = $a0
    if (n == 0) return 0;
    return isEven(n - 1);
}
```

Answer:

```
isEven:
    bne $a0, $zero, even_recurse
    addi $v0, $zero, 1
    jra $ra

even_recurse:
    addi $sp, $sp, -8
    sw $ra, 4($sp)
    sw $a0, 0($sp)
    addi $a0, $a0, -1
    jal isOdd
    lw $a0, 0($sp)
    jra $ra

isOdd:
    bne $a0, $zero, odd_recurse
    addi $v0, $zero, 0
    jra $ra

odd_recurse:
    addi $sp, $sp, -8
    sw $ra, 4($sp)
    sw $a0, 0($sp)
    addi $a0, $a0, -1
    jal isEven
    lw $a0, 0($sp)
    jra $ra
```

Question 12 (10 marks)

Translate the following C code into MIPS assembly.

```
int ackermann(int m, int n) {      // m = $a0, n = $a1
    if (m == 0) {
        return n + 1;
    }
    if (n == 0) {
        return ackermann(m - 1, 1);
    }
    return ackermann(m - 1, ackermann(m, n - 1));
}
```

Answer:

ackermann:

```
bne $a0, $zero, if
addi $v0, $a1, 1
jra $ra
```

if:

```
bne $a1, $zero, recursive
addi $sp, $sp, -12
sw $ra, 8($sp)
sw $a0, 4($sp)
sw $a1, 0($sp)
addi $a0, $a0, -1
jal ackermann
lw $a0, 4($sp)
lw $a1, 0($sp)
lw $ra, 8($sp)
addi $sp, $sp, 12
jra $ra
```

recursive:

```
addi $sp, $sp, -12
sw $ra, 8($sp)
sw $a0, 4($sp)
sw $a1, 0($sp)
addi $a1, $a1, -1
jal ackermann
addi $a1, $v0, 0
lw $a0, 4($sp)
addi $a0, $a0, -1
jal ackermann
lw $ra, 8($sp)
addi $sp, $sp, 12
jra $ra
```

Question 13 (10 marks)

Translate the following C code into MIPS assembly.

```
int recArraySum(int arr[], int left, int right) {
    // arr = $a0, left = $a1, right = $a2
    if (left > right) {
        return 0;
    }
    if (left == right) {
        return arr[left];
    }
    int mid = (left + right) / 2;    // mid = $t0
    return recArraySum(arr, left, mid)
        + recArraySum(arr, mid + 1, right);
}
```

Answer:

recArraySum:

```
slt $t0, $a2, $a1
bne $t0, $zero, if
addi $v0, $zero, 0
jra $ra
```

if:

```
bne $a1, $a2, recursive
sll $t0, $a1, 2
add $t1, $t0, $a0
lw $t2, 0($t1)
add $v0, $zero, $t2
jra $ra
```

recursive:

```
add $t1, $a1, $a2
srl $t0, $t1, 1
addi $sp, $sp, -20
sw $v0, 16($sp)
sw $a0, 12($sp)
sw $a1, 8($sp)
sw $a2, 4($sp)
```

```
sw $t0, 0($sp)
addi $a2, $t0, 0
jal recArraySum
sw $v0, 16($sp)
lw $t0, 0($sp)
lw $a2, 4($sp)
addi $a1, $t0, 1
lw $t2, 16($sp)
jal recArraySum
lw $a1, 8($sp)
```

```
add $v0, $t2, $v0
addi $sp, $sp, 20
jra $ra
```



Question 14 (10 marks)

Translate the following C code into MIPS assembly.

```
int countPaths(int m, int n) {          // m = $a0, n = $a1
    if (m == 0 || n == 0) {
        return 1;
    }
    if (m == n) {
        return 2 * countPaths(m - 1, n);
    }
    return countPaths(m - 1, n) + countPaths(m, n - 1);
}
```

Answer:

countPaths:

beq \$a0, \$zero, if $m == 0$ beq \$a1, \$zero, if $n == 0$ beq \$a1, \$a2, second-if $m == n$

addi \$sp, \$sp, -16

sw \$ra, 12(\$sp)

sw \$a0, 8(\$sp) m sw \$a1, 4(\$sp) n

sw \$v0, 0(\$sp)

addi \$a1, \$a1, -1 $n-1$ jal countPaths $\text{countPaths}(m, n-1)$ sw \$v0, 0(\$sp) $\text{store countPaths}(m, n-1)$

add \$t0, \$v0, \$zero

lw \$a1, 4(\$sp) $\text{restore } n$ addi \$a0, \$a0, -1 $m-1$ jal countPaths $\text{countPaths}(m-1, n)$

lw \$a0, 8(\$sp)

lw \$a1, 4(\$sp)

lw \$ra, 12(\$sp)

add \$v0, \$t0, \$v0 $\text{countPaths}(m-1, n) + \text{countPaths}(m, n-1)$

jr \$ra

if:

addi \$v0, \$zero, 1

jr \$ra return 1

second-if:

addi \$sp, \$sp, -12

sw \$ra, 8(\$sp)

sw \$a0, 4(\$sp) m sw \$a1, 0(\$sp) n addi \$a0, \$a0, -1 $m-1$

addi \$t0, \$zero, 2

jal countPaths $\text{countPaths}(m-1, n)$

lw \$ra, 8(\$sp)

lw \$a0, 4(\$sp)

lw \$a1, 0(\$sp)

mul \$v0, \$v0, \$t0 $2 * \text{countPaths}(m-1, n)$

jr \$ra

Question 15 (10 marks)

Translate the following C code into MIPS assembly.

```
struct Node {
    int val;
    struct Node *next;
};

int recListMax(struct Node *head) {      // head = $a0
    if (head->next == 0) {
        return head->val;
    }
    int restMax = recListMax(head->next); // $t0
    return (head->val > restMax) ? head->val : restMax;
}
```

Answer:

recListMax:

lw \$t0, 4(\$a0) *head → next*

bne \$t0, \$zero, recursion *head → next != 0*

lw \$v0, 0(\$a0) *head → val*

jra \$ra

recursion:

addi \$sp, \$sp, -8

sw \$ra, 4(\$sp)

sw \$a0, 0(\$sp) *head*

lw \$t0, 4(\$a0) *head → next*

jal recListMax

addi \$t1, \$v0, 0 *nextMax = recListMax(head → next)*

lw \$t2, 0(\$a0) *head → value*

bne \$t1, \$t2, check

addi \$v0, \$t2, 0 *return head → value*

lw \$ra, 4(\$sp), lw \$a0, 0(\$sp), addi \$sp, \$sp, 8, jra \$ra

check:

addi \$v0, \$t1, 0 *return nextMax*

jra \$ra

Section D Structs, Bitwise Ops & Mixed Logic

Question 16 (10 marks)

Translate the following C code into MIPS assembly.

```
struct Rect {
    int x, y;
    int width, height;
};

int overlapArea(struct Rect *a, struct Rect *b) {
    // a = $a0, b = $a1
    int left    = (a->x > b->x) ? a->x : b->x;           // $t0
    int right   = (a->x + a->width < b->x + b->width)
                  ? a->x + a->width : b->x + b->width;   // $t1
    int top     = (a->y > b->y) ? a->y : b->y;           // $t2
    int bottom  = (a->y + a->height < b->y + b->height)
                  ? a->y + a->height : b->y + b->height; // $t3

    if (right <= left || bottom <= top) {
        return 0;
    }
    return (right - left) * (bottom - top);
}
```

Answer:



Question 17 (10 marks)

Translate the following C code into MIPS assembly.

```
unsigned int reverseBits(unsigned int n) {          // n = $a0
    unsigned int result = 0;                        // result = $t0
    int i;                                          // i = $t1
    for (i = 0; i < 32; i++) {
        result = result << 1;
        if ((n & 1) == 1) {
            result = result | 1;
        }
        n = n >> 1;
    }
    return result;
}
```

Answer:



Question 18 (10 marks)

Translate the following C code into MIPS assembly.

```
int matrixDiagDiff(int mat[4][4]) {           // mat base addr = $a0
    int mainSum = 0, antiSum = 0;             // $t0, $t1
    int i;                                     // $t2
    for (i = 0; i < 4; i++) {
        mainSum += mat[i][i];
        antiSum += mat[i][3 - i];
    }
    if (mainSum > antiSum) {
        return mainSum - antiSum;
    }
    return antiSum - mainSum;
}
```

Answer:



Question 19 (10 marks)

Translate the following C code into MIPS assembly.

```
struct Node {
    int val;
    struct Node *next;
};

int listSumEven(struct Node *head) {           // head = $a0
    int sum = 0;                               // sum = $v0
    struct Node *cur = head;                  // cur = $t0
    while (cur != 0) {
        if ((cur->val & 1) == 0) {
            sum += cur->val;
        }
        cur = cur->next;
    }
    return sum;
}
```

Answer:



```
void strcpy (char x[], char y[])
```

```
{ int i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

- Addresses of x, y in \$a0, \$a1
- i in \$s0

— End of Exam —

strcpy:

addi \$sp, \$sp, -4

sw \$s0, 0(\$sp)

add \$s0, \$zero, \$zero *i = 0*

while:

add \$t1, \$s0, \$a1 *y[i]*

lbv \$t2, 0(\$t1)

add \$t3, \$s0, \$a0 *x[i]*

sb \$t2, 0(\$t3)

beq \$t2, \$zero, end

add \$s0, \$s0, 1

end:

lw \$s0, 0(\$sp)

addi \$sp, \$sp, 4

jr \$ra